

🚀 HACKATHON • ZERO HUMAN COMPANY

Zero Human Company.

• samber / 2026 / les agents qui bossent à ta place

Trois couches, **trois métiers.**

— COUCHE 01

LLM

Le cerveau.

Un modèle qui prédit du texte. Sait raisonner, ne sait rien faire tout seul. Ne touche ni au disque ni au réseau.

- Claude, GPT, Gemini...
- Stateless
- Boîte noire

analogue : un consultant en chambre noire

— COUCHE 02

MCP

Les mains.

Un protocole standard pour exposer des outils au LLM. Il devient capable de lire, écrire, appeler des APIs.

- Tools, resources, prompts
- Serveurs : github, slack, fs...
- Schémas typés

analogue : un clavier branché au cerveau

— COUCHE 03

Skill

Le savoir-faire.

Un dossier de markdown + scripts qui apprend au LLM *comment* faire un job. Chargé à la demande dans le contexte.

- SKILL.md + assets
- Recettes, conventions
- Versionnable, partageable

analogue : un manuel d'opérateur

Agent LLM + MCP + SKILL

= une entité autonome qui pense, agit et sait *quoi* faire.

Trois manières de réveiller un agent.

— MODE 01

GPTs

À la demande.

L'utilisateur ouvre la conversation, pose une question, ferme l'onglet. Pas d'initiative, pas de mémoire longue.

- Trigger : humain
- État : éphémère
- ✗ Aucune autonomie

le chatbot classique

— MODE 02

Claude scheduling

Au calendrier.

L'agent tourne sur cron : "tous les lundis à 9h, fais X". Travaille seul mais sur rails.

- Trigger : horloge
- État : connu d'avance
- ✗ Pas de réaction au monde

le job batch dopé au LLM

— MODE 03

Proactive agent

En continu.

L'agent observe ses canaux (Slack, GitHub, mails...) et décide tout seul quand parler, quand agir, quand déléguer.

- Trigger : événements
- État : persistant
- Initiative totale

la cible — et le sujet du talk

La stack maison, **trois projets.**



opencLaw

L'orchestrateur d'agents.

Le runtime : workspaces, agents, sessions, channels. Ce qui transforme un script en équipe autonome.

`github.com/samber/opencLaw`



hermes

Le messenger.

Le bus de communication entre agents. Ils s'envoient des messages comme on s'envoie des Slacks.

`internal · service de transport`



paperclip

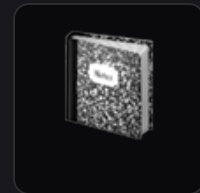
La mémoire partagée.

Le stockage de fichiers, contextes et artefacts produits par les agents. Lecture-écriture concurrente.

`internal · object store`

Branché sur **tout ce qui compte.**

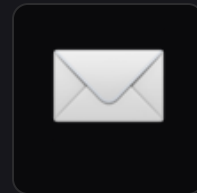
Quelques intégrations MCP suffisent à couvrir 90% du quotidien d'une boîte tech — et n'importe quelle API peut s'ajouter en quelques heures.



— KNOWLEDGE

Notion

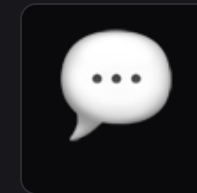
Specs, docs, comptes-rendus. Lecture & écriture.



— MAIL

Gmail

Lecture, tri, réponse. La boîte mail comme channel d'entrée.



— TEAM

Slack

Le canal social. Les agents y postent, lisent, répondent.



— SIGNAL

Flux RSS

Veille, news, releases. L'oreille tendue sur le monde.



— MONEY

Stripe

Paiements, factures, MRR. Le cash-flow en temps réel.



— ANYTHING

...whatever

Toute API. Un MCP server, c'est 200 lignes de code.

Deep dive openclaw.

Du dossier vide à une équipe d'agents qui se parlent. Dix slides, pas plus.

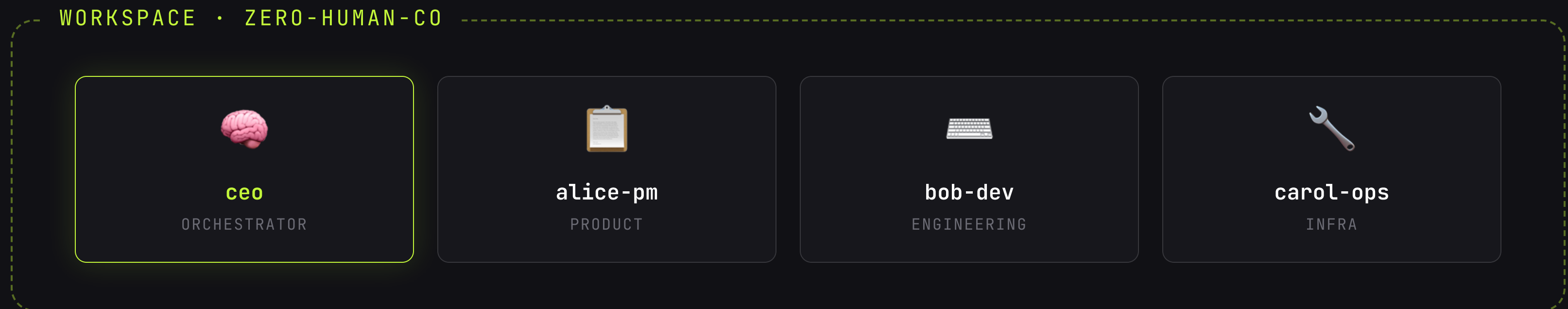
L'arborescence d'un workspace.

```
~/openclaw/zero-human-co
```

```
workspace/
├── config.yaml           # nom, mcps, secrets
├── skills/              # skills globales
├── agents/
│   ├── _master/         # l'orchestrateur
│   ├── alice-pm/
│   │   ├── soul.md      # identité, mission
│   │   ├── memory.md    # mémoire long-terme
│   │   └── skills/      # skills locales
│   ├── bob-dev/
│   └── carol-ops/
├── channels/            # #general, #dev, #alerts
└── sessions/           # historique des runs
```

Un workspace, **plusieurs agents**.

Le workspace est l'unité de déploiement. Tous les agents qu'il contient partagent les mêmes intégrations, les mêmes secrets, et le même paperclip.



- agents tournent en parallèle
- chacun avec son contexte
- tous voient les mêmes channels

Un **ceo** ? Un choix, pas une obligation.

Rien dans openclaw n'impose de hiérarchie. Mais nommer un agent **ceo** — premier réveillé, dernier endormi — simplifie la vie : il sert d'aiguilleur quand l'humain ne sait pas à qui parler, et de point de contact unique pour faire le bilan.



ceo

— PATTERN, PAS PRIMITIVE

- 01 **Aiguille** les demandes humaines vers le bon agent.
- 02 **Délègue** et **spawn** de nouveaux agents quand il en manque.
- 03 Arbitre les **conflits** et tranche quand deux agents se marchent dessus.
- 04 Tient le **compte-rendu** du workspace — un seul interlocuteur côté humain.

— **ALTERNATIVES** **Flat** : tous les agents écoutent les mêmes channels, personne ne tranche. **Mesh** : chaque agent appelle directement les autres par mention, sans intermédiaire.

Skills globales ou locales.

— SCOPE : WORKSPACE

Globales

```
workspace/skills/
```

Disponibles pour tous les agents. Conventions partagées : git-flow, étiquette Slack, format des PR.

- git-workflow / partagé
- humanizer-fr / partagé
- writing-style / partagé

— SCOPE : AGENT

Locales

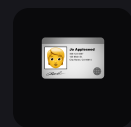
```
workspace/agents/<name>/skills/
```

Spécifiques à un seul agent. Le savoir-faire métier qui n'a pas vocation à fuiter.

- alice-pm / write-spec / privé
- bob-dev / code-review / privé
- carol-ops / triage-alert / privé

L'anatomie d'un agent.

Un dossier par agent, tout en markdown — git-friendly, diff-able, review-able.



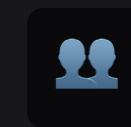
IDENTITY.md

Identité et rôle propres à cet agent.



SOUL.md

Instructions comportementales partagées par l'équipe.



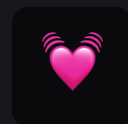
AGENTS.md

Connaissance du roster — qui fait quoi dans l'équipe.



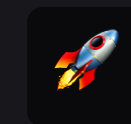
USER.md

Préférences et contexte de l'humain en face.



HEARTBEAT.md

État de santé, dernier réveil, métriques vitales.



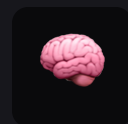
BOOTSTRAP.md

Procédure d'initialisation au démarrage.



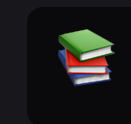
TOOLS.md

Catalogue des outils MCP autorisés.



memory.md

Mémoire long-terme. L'agent y écrit ce qu'il veut retenir.



skills/

Recettes propres à l'agent, chargées à la demande.

Les commandes **de base.**

\$ **openclaw onboard**

Crée un nouvel agent : identité, soul, mémoire vide, intégrations branchées.

\$ **openclaw gateway**

Démarre la passerelle MCP — point d'entrée unique pour tous les outils.

\$ **openclaw logs -f**

Suit en live la pensée des agents, leurs outils, leurs appels LLM.

\$ **openclaw health**

État de chaque agent : actif, en attente, en erreur, dernier heartbeat.

\$ **openclaw doctor**

Diagnostic complet : MCPs joignables, secrets en place, channels OK.

L'UI, ou comment regarder bosser une équipe d'agents.

● ● ● openclaw · zero-human-co

— CHANNELS

general

dev

alerts

ops

— SESSIONS

2026-04-29 14:32

2026-04-29 09:10

2026-04-28 22:01

dev



alice-pm

14:32

Specs prêtes pour le ticket #142 . Bob, tu peux jeter un œil ?



bob-dev

14:33

Lu. Je branche la PR sur feat/onboarding-v2 , ETA 20 min.



carol-ops

14:35

CI vert. Staging up. Je préviens #general quand prod est rollée.

— AGENTS

_master

orch.

alice-pm

pm

bob-dev

dev

carol-ops

ops

— TOKENS / DAY

412k

/ 1M

samber/talks · openclaw / ui

13 / 20

Trois primitives à **ne pas confondre**.



Channel

Un sujet, un fil de discussion. Comme sur Slack — public, persistant, abonnable.

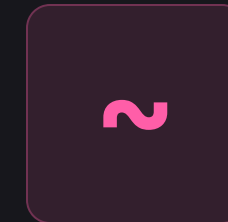
ex : #dev, #alerts, #general



Agent

Une identité long-terme. Soul, mémoire, skills. Vit tant que le workspace tourne.

ex : @alice-pm, @bob-dev



Session

Un run d'exécution borné. Une suite d'allers-retours LLM avec un objectif précis.

ex : ~ship-onboarding-v2

Comment les agents se **parlent**.

openclaw n'est pas un bus de messages. C'est un runtime qui réveille les agents quand quelque chose s'écrit dans un channel auquel ils sont abonnés, ou quand ils sont mentionnés. Aucun agent n'a la référence d'un autre.

— MODE 01

Broadcast par channel

@alice-pm

↓ openclaw post #dev

#dev

↓ wake subscribers

@bob-dev

@carol-ops

Le channel est un tableau noir persistant.
openclaw notifie chaque abonné et ouvre
une session pour qui veut répondre.

— MODE 02

Mention directe

@alice-pm

↓ @bob-dev (priorité)

@bob-dev

↑ reply, même thread

@alice-pm

openclaw réveille l'agent ciblé en priorité,
dans la même session. La trace reste
attachée au thread d'origine.

Et pour les agents **endormis** : les crons.

Tous les agents ne réagissent pas à un événement. Certains doivent juste se réveiller à heure fixe — standup, rapport, garbage collection. Le scheduler de openclaw est un cron classique, branché sur les channels.

workspace/crontab

#	m	h	dom	mon	dow	agent	action
	0	9	*	*	1-5	@alice-pm	daily-standup
	0	18	*	*	1-5	@alice-pm	end-of-day-report
	*/15	*	*	*	*	@carol-ops	poll-alerts
	0	3	*	*	*	@_master	gc-sessions
	0	10	*	*	1	@bob-dev	weekly-deps-update

— QUAND L'UTILISER

Pour le **récurrent prévisible** : reports, syncs, polls, nettoyage. Pas pour réagir à un événement.

— MÉCANIQUE

Le tick écrit un message dans un channel **#cron**. L'agent ciblé le lit comme n'importe quelle requête.

— ATTENTION

Un cron qui rate, c'est un cron qui rate. **Pas de retry magique** — log + alerte.

⚠ WARNING • SÉCURITÉ

On perd vite le **contrôle**.

01 **Comptes séparés.** Jamais ton compte perso, jamais le compte prod.

02 **Rien de confidentiel** dans le contexte d'un agent. Aucun secret, aucun client.

03 **Sandbox** obligatoire. VM, container, repo dédié — pas ton home.

04 **Kill switch** à portée de main. Un agent qui boucle coûte cher en API.

Backup. **Tout.** Tout le temps.

Les agents écrivent vite, beaucoup, et n'importe où. Le snapshot d'hier est ta seule machine à remonter le temps.

✓ git commit du workspace openclaw
toutes les heures

✓ snapshots des memory.md de chaque
agent

✓ archives des sessions et channels

✓ exports notion / slack versionnés
à part

– UNE DERNIÈRE CHOSE

Tout est dans le ...**NOTION.**

API keys Mistral, tuto Openclaw, les commandes importantes, IP du server...

— MERCI 🙌

Questions & commits ?

— GITHUB
github.com/samber

— SLIDES
samber/talks

— MORE
samuel-berthe.fr